UTEC
Universidad Tecnológica

Tablas Hash (HashTable)

Una tabla hash es una estructura de datos que asocia claves (keys) con valores (values) utilizando una función hash para calcular un índice en una matriz de buckets o contenedores, desde el cual se puede encontrar el valor deseado. Es altamente eficiente para operaciones de búsqueda, inserción y eliminación.

Características principales:

- Acceso Rápido: Las operaciones de búsqueda, inserción y eliminación tienen un tiempo promedio de complejidad O(1).
- Función Hash: Utiliza una función hash para transformar la clave en un índice dentro de un array.
- Resolución de Colisiones: Implementa técnicas para manejar situaciones en las que dos claves diferentes se mapean al mismo índice.

Operaciones básicas:

1. Inserción (put): Añade un par clave-valor a la tabla hash.
2. Búsqueda (get): Recupera el valor asociado con una clave.
3. Eliminación (remove): Elimina el par clave-valor asociado con una clave.
4. Contiene Clave (containsKey): Verifica si una clave está en la tabla.
5. Tamaño (size): Devuelve el número de pares clave-valor en la tabla.

Uso recomendado:

- Almacenamiento y Recuperación Rápida: Ideal para aplicaciones que requieren acceso rápido a datos, como bases de datos, cachés, e índices.
- Mapeo de Claves a Valores: Útil en situaciones donde se necesitan asociaciones rápidas, como tablas de símbolos en compiladores, diccionarios, y sistemas de directorios.

Implementación y ejemplos en Java:

Implementación básica:

```
import java.util.HashMap;
import java.util.Map;

public class EjemploTablaHash {
```

UTEC
Universidad Tecnológica

```
    public static void main(String[] args) {
        // Crear una tabla hash
        Map<String, Integer> tablaHash = new HashMap<>();

        // Insertar elementos en la tabla hash
        tablaHash.put("uno", 1);
        tablaHash.put("dos", 2);
        tablaHash.put("tres", 3);

        // Buscar elementos en la tabla hash
        int valor = tablaHash.get("dos"); // Devuelve 2
        System.out.println("El valor asociado con 'dos' es: " + valor);

        // Eliminar un elemento de la tabla hash
        tablaHash.remove("tres");

        // Verificar si una clave está en la tabla hash
        boolean contieneClave = tablaHash.containsKey("uno"); // true
        System.out.println("¿La tabla contiene la clave 'uno'? " +
contieneClave);

        // Obtener el tamaño de la tabla hash
        int tamaño = tablaHash.size(); // 2
        System.out.println("El tamaño de la tabla hash es: " + tamaño);
    }
}
```

Funciones Hash y Colisiones:

Función Hash:

La función hash es una función que toma una clave y devuelve un número entero (hash code) que se usa como índice para almacenar el valor correspondiente en la tabla hash. En Java, el método `hashCode()` de la clase `Object` se utiliza para obtener el hash code de un objeto.

Colisiones:

Las colisiones ocurren cuando dos claves diferentes producen el mismo hash code y se mapean al mismo índice de la tabla hash. Existen varias técnicas para resolver colisiones:

UTEC
Universidad Tecnológica

Resolución de colisiones:

1. Encadenamiento (Chaining):
   Consiste en almacenar múltiples elementos en el mismo índice usando una estructura de datos secundaria, como una lista enlazada.

```
import java.util.LinkedList;

class HashTableWithChaining<K, V> {
    private LinkedList<Entry<K, V>>[] table;
    private int size;

    public HashTableWithChaining(int capacity) {
        table = new LinkedList[capacity];
        for (int i = 0; i < capacity; i++) {
            table[i] = new LinkedList<>();
        }
    }

    static class Entry<K, V> {
        K key;
        V value;

        public Entry(K key, V value) {
            this.key = key;
            this.value = value;
        }
    }

    private int hash(K key) {
        return Math.abs(key.hashCode() % table.length);
    }

    public void put(K key, V value) {
        int index = hash(key);
        for (Entry<K, V> entry : table[index]) {
            if (entry.key.equals(key)) {
                entry.value = value;
                return;
            }
```

```
            }
            table[index].add(new Entry<>(key, value));
            size++;
        }

        public V get(K key) {
            int index = hash(key);
            for (Entry<K, V> entry : table[index]) {
                if (entry.key.equals(key)) {
                    return entry.value;
                }
            }
            return null;
        }

        public void remove(K key) {
            int index = hash(key);
            for (Entry<K, V> entry : table[index]) {
                if (entry.key.equals(key)) {
                    table[index].remove(entry);
                    size--;
                    return;
                }
            }
        }

        public boolean containsKey(K key) {
            int index = hash(key);
            for (Entry<K, V> entry : table[index]) {
                if (entry.key.equals(key)) {
                    return true;
                }
            }
            return false;
        }

        public int size() {
            return size;
        }
    }
```

UTEC
Universidad Tecnológica

2. Direccionamiento Abierto (Open Addressing):

En este método, cuando ocurre una colisión, se busca otro índice vacío en la tabla utilizando una secuencia de búsqueda. Los métodos comunes incluyen:

- Lineal (Linear Probing): Se revisan los siguientes índices secuenciales hasta encontrar un espacio vacío.
- Cuadrático (Quadratic Probing): Se revisan índices en intervalos crecientes cuadráticamente hasta encontrar un espacio vacío.
- Doble Hashing (Double Hashing): Utiliza una segunda función hash para determinar los intervalos de búsqueda.

Ejemplo de uso práctico:

Diccionario de Traducción:
Imagina que estás creando un diccionario de traducción entre dos idiomas. Una tabla hash sería ideal para mapear palabras de un idioma a otro, permitiendo una búsqueda rápida y eficiente.

```
import java.util.HashMap;
import java.util.Map;

public class DiccionarioTraduccion {
    public static void main(String[] args) {
        Map<String, String> diccionario = new HashMap<>();

        // Añadir palabras al diccionario
        diccionario.put("casa", "house");
        diccionario.put("gato", "cat");
        diccionario.put("perro", "dog");

        // Buscar traducción de una palabra
        String traduccion = diccionario.get("gato");
        System.out.println("La traducción de 'gato' es: " + traduccion);

        // Eliminar una palabra del diccionario
        diccionario.remove("perro");

        // Verificar si una palabra está en el diccionario
        boolean contienePalabra = diccionario.containsKey("casa");
        System.out.println("¿El diccionario contiene la palabra 'casa'?
```

```
" + contienePalabra);

        // Obtener el tamaño del diccionario
        int tamaño = diccionario.size();
        System.out.println("El tamaño del diccionario es: " + tamaño);
    }
}
```

Las tablas hash en Java, especialmente con la clase `HashMap`, proporcionan una manera eficiente y flexible de manejar asociaciones clave-valor, siendo esenciales para una amplia gama de aplicaciones en el desarrollo de software.